

CHAP 5: (1) 자료의 입력과 출력

입력 함수 `input()`,

정수를 입력받기, 실수를 입력받기

출력 함수 `print()`

문자열 포매팅 (`Formatting`)

문자열 포매팅 - `str.format()` 사용

입력 함수 input()

- input() 함수 - 키보드로부터 값을 입력 받을 때 사용.
입력 값을 문자열로 반환한다.
- 함수 사용

```
변수 = input('문자열')
```

- 함수의 인수 '문자열'은 일종의 프롬프트이다.
 - 사용자에게 안내 문구 역할을 함.
 - '문자열'이 화면에 출력되고, 사용자로부터 입력 기다림!!!
- 사용자가 키보드로 값을 입력하고 Enter 키를 누름.
- 함수는 입력 값을 문자열로 반환, 이 값을 변수에 할당한다.

```
>>> name = input('이름 입력:')  
이름 입력: 
```

input() 함수의 인수, 즉 괄호 안의 문자열이 화면에 출력되고, 그 뒤에 커서가 깜빡임.
여기에서 입력한다.

입력 함수 input()

```
>>> name = input("이름 입력:") Enter
```



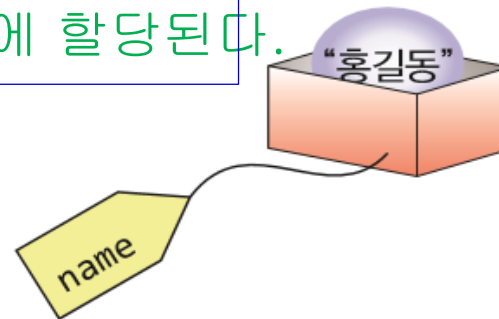
여기에서
키보드로 값을 입력하고 Enter를 친다



```
>>> name = input("이름 입력:") Enter
이름 입력: 홍길동 Enter
>>>
>>> name Enter
'홍길동'
>>>
```



키보드로 입력 후 Enter를 치면,
입력 값이 문자열 형태로 반환되어
변수에 할당된다.

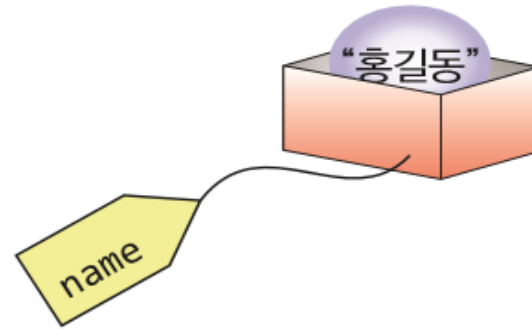


입력 함수 input()

- (예) 소스 파일로 저장하여 실행

test.py

```
name = input("이름 입력:")  
print(name)  # name 출력
```



☺ 실행화면 (Python Shell)

```
>>>  
===== RESTART: C:\test.py =====  
이름 입력: 홍길동 Enter  # input() 실행  
홍길동                  # 변수 name 출력  
>>>
```



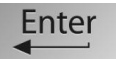
입력 함수 input()

- 프로그램을 실행할 때마다 값을 받으려면, **input()** 함수를 사용.
 - 사용자가 키보드에서 입력한 값을 받아서, 문자열로 반환.
 - 다른 자료형이 필요하다면, 자료형 변환을 해야한다.

- **사용법1: 프롬프트를 띄우지 않고 사용자 입력 받기**

```
변수 = input()
```

- **변수**: 사용자가 키보드에서 입력한 값을 할당 받는 변수.
- 첫번째 컬럼에서 커서가 깜빡이며 입력을 기다림.
- 사용자가 값을 입력하고 **Enter** 키를 누르면, 그 값이 변수에 할당됨.
문자열 값으로 할당된다.
- 예)

```
>>> name = input()  # 안내문구 사용 안함.  
Gildong  # 사용자가 키보드에서 입력한 값.  
>>> name   
'Gildong'
```

입력 함수 input()

- 사용법2 : 프롬프트를 띄워서 사용자 입력 받기

```
변수 = input('문자열')
```

- 먼저 안내 문구(프롬프트)를 띄우고,
그 다음 컬럼에서 커서가 깜빡이며 입력을 기다림.
- '문자열': 안내 문구.
 - 입력 받기 전에 화면으로 출력되는 메시지.
 - 일종의 프롬프트(prompt).
 - 사용자에게 안내 문구 역할을 함.
 - '문자열'이 출력되고, 사용자로부터 입력 기다림!!!
- 사용자가 값을 입력하고 **Enter** 키를 누르면, 그 값이 변수에 할당됨.
- 예)

```
>>> name = input('first name?? ')   
first name?? Gildong   
>>> name   
'Gildong'
```

입력 함수 input()

- 두 개의 **string**을 입력 받기
 - 문자열 자료형의 메소드 **split()**을 사용하여 분리.
 - 예) 이름을 한꺼번에 2개 입력 받기
 - **input()**으로 입력 받은 후 **split()** 함수를 사용하여 분리한다.

```
>>> n = input("Enter two names: ") Enter
```

```
Enter two names: Thom Bob Enter
```

```
>>> n Enter
```

```
'Thom Bob'
```

```
>>> n1, n2 = n.split()
```

```
>>> n1 Enter
```

```
'Thom'
```

```
>>> n2 Enter
```

```
'Bob'
```

동일한 작업

```
>>> n1 = input("Enter name: ") Enter
```

```
Enter name: Thom
```

```
>>> n2 = input("Enter name: ") Enter
```

```
Enter name: Bob
```

```
>>> n1
```

```
'Thom'
```

```
>>> n2
```

```
'Bob'
```

입력 함수 input()

- 예) 이름을 한꺼번에 2개 입력 받기
 - input() 함수의 반환값이 문자열이므로, 문자열처럼 취급 가능.

```
>>> n = input("Enter two names: ")   
Enter two name : Thom Bob   
>>> n1, n2 = n.split()  
>>>
```

입력과 분리 작업을
한 문장으로 처리 가능

```
>>> n1, n2 = input("Enter two names: ").split()   
Enter two names: Thom Bob   
>>> n1  
'Thom'  
>>> n2  
'Bob'
```

input()의 반환 값이 문자열.
따라서
이 함수를 문자열처럼 취급하여
split()함수를 사용 가능.

정수를 입력 받기

- `input()` 함수에서 정수를 입력 받기
 - `input()` 함수를 통해 입력 받은 값은 문자열. 정수로 형 변환해야 함.
 - 정수 형태의 문자열 → 정수로 형 변환
 - `int()` 함수를 사용하여 정수로 형 변환.
- 예) 키보드 입력 값(문자열) → 정수로 형 변환

```
>>> s = input() Enter
10 Enter
>>> s Enter # '10' 문자열 값이 반환, 변수에 할당
'10'
>>>
>>> s = int(s) Enter #10 (정수로 형 변환)
>>> s Enter
10
>>>
```

정수를 입력 받기

- 예) 나이를 입력 받아, 출생년도 계산하기

```
>>> age = input("Enter your age: ")
Enter your age: 22 
>>> age                                # '22' (입력 값은 문자열)
'22'
>>> age = int(age)                     # '22' → 22 (정수로 형 변환)
>>> age
22
>>>
```

```
>>> bornYear = 2022-age+1
>>> print( bornYear, '년생', age, '살' )
2001년생22살
>>> print( str(bornYear) + '년생', str(age) + '살' )
2001년생22살                                문자열 연결
```

정수를 입력 받기

- 응용 예) 커피가게 매출 계산하기..
다음과 같은 커피 메뉴가 있을 때, 매출액을 계산해보자.

아메리카노- 2000원
카페라떼- 3000원
카푸치노- 3500원



☺실행 화면(Python Shell 창)

아메리카노 판매 개수? 10 Enter
카페라떼 판매 개수? 20 Enter
카푸치노 판매 개수? 30 Enter
총 매출은 185000 입니다



input()함수 사용. 키보드에서 판매 개수를 입력 받음.

메뉴별 판매 개수를 키보드로 받아들이기

```
a = input("아메리카노 판매 개수? ") # a= '10'  
latte = input("카페라떼 판매 개수? ") # latte= '20'  
capu = input("카푸치노 판매 개수? ") # capu= '30'
```

정수를 입력 받기

☺실행 화면(Python Shell 창)

test.py

```
# 메뉴별 가격을 변수에 할당  
price1 = 2000  
price2 = 3000  
price3 = 3500
```

```
# 키보드에서 판매 개수 입력 받기
```

```
a = input("아메리카노 판매 개수? ")  
latte = input("카페라떼 판매 개수? ")  
capu = input("카푸치노 판매 개수? ")
```

```
# a= '10'  
# latte= '20'  
# capu= '30'
```

```
# 판매 개수를 숫자로 바꿈
```

```
a = int(a)      # 정수로 변환 '10' → 10  
latte = int(latte)  # 정수로 변환  
capu = int(capu)  # 정수로 변환
```

```
# 메뉴 가격, 판매 개수를 사용한 수식으로 매출액 계산
```

```
total = a * price1  
total += latte * price2  
total += capu * price3  
print("총 매출은", total, "입니다.")
```

아메리카노 판매 개수? 10 Enter
카페라떼 판매 개수? 20 Enter
카푸치노 판매 개수? 30 Enter
총 매출은 185000 입니다

정수를 입력 받기

- 정수를 3개 함께 입력 받기
 - `input()` 함수로 입력 받고, 분리하여, 형변환한다.

```
>>> num = input("3개 메뉴의 판매개수?? ")
3개 메뉴의 판매개수?? 10 20 30
>>> num
'10 20 30'
>>> n1, n2, n3 = num.split() # 문자열을 3개로 분리
>>> n1
'10'
>>> n2
'20'
>>> n3
'30'
>>> n1 = int(n1) # 10. 정수로 형변환
>>> n2 = int(n2) # 20
>>> n3 = int(n3) # 30
```

출력 함수 print()

- print (값1, 값2, 값3)

- 모니터 화면에 값을 출력하기 위해 사용.
- 함수의 인수는 출력하고자 하는 데이터.
- 함수의 인수를 문자열로 바꾸어 출력한다.
항상 끝에 줄 바꿈 문자 '\n'가 출력되어 줄 바꿈이 일어남.
- 콤마(,)로 구분된 인수들 사이에 공백 1칸이 자동으로 추가되어 출력

```
>>> print ('Hello') ; print('???')
Hello
???
```

- 예) 값 출력하기

```
print ('Hello')  #(1)
print('??? ')
```

😊 출력 화면

```
Hello
???
```

(1) print ("Hello")

😊 출력 화면

```
Hello
```

다음 출력을
위한 커서 위치
'Hello' 뒤에 '\n'가 출력됨.

출력 함수 print()

- 콤마는 공백 1칸이 추가되어 출력됨. 띄어쓰기를 한다.

- 예)

```
>>> print( 2000, '년생', 22, '살' )  
2000 ■년생 ■22 ■살
```

빈칸이 추가됨

- 출력 함수 print()의 인수는
정수 2000, 문자열 '년생', 정수 22, 문자열 '살'
- 지정된 인수들을 함께 차례로 출력
- 인수 사이의 콤마(,)는 공백 1칸이 추가되어 띄어쓰기 한다.
- 연결 연산자(+)로 문자열을 연결 할 경우 공백없이 연결되어 출력.

```
>>> print( str(2000)+'년생', str(22)+'살' )  
2000년생 ■22살
```

출력 함수 print()

- `print()` 함수의 줄 바꿈을 방지하려면?
 - `end` 인자를 사용하여, 새로운 끝 문자를 지정.
디폴트 끝 문자 '`\n`' (줄 바꿈 문자)를 대체.
 - 사용법
`print ( 출력할 objects, end= '문자열' )`
끝 문자
 - 예) 끝 문자로 지정하여 같은 줄에 출력.

```
print ( 'Hello', end=" ) #빈문자열을 끝 문자로 지정
print ( "???" )
```

😊 출력 화면

Hello???

출력 함수 print()

- 예) print()의 끝문자를 공백문자로 변경해서, 한 줄에 출력하기

test.py

```
print( 'Hello', end=' ' ) #(1)  
print( "???" )
```

😊 출력 화면

Hello ■???

(1) print ("Hello", end=" ")

실행화면

Hello ■

다음 출력을 위한 커서 위치
'Hello' 뒤에 끝문자 ' ' 출력

출력 함수 print()

- 예) 끝 문자의 기본값 '\n'

test1.py

```
print('Hello')
print('Hannah')
print('*')
print('*')
print('*')
print()
print('*')
print('*')
print('*')
```

실행 화면

```
Hello
Hannah
*
*
*
*
*
*
```

- 예) 끝 문자를 지정하여 출력하기

test2.py

```
print('Hello', end=' ') # 끝문자 ' '
print('Hannah')
print('*', end='') # 끝문자 ''
print('*', end='')
print('*', end='')
print() # 줄바꿈
print('*', end=' ') # 끝문자 ' '
print('*', end=' ')
print('*', end=' ')
```

실행 화면

```
Hello Hannah
***
* * *
```

문자열 포매팅(Formatting)

- 문자열 안에 다른 데이터를 삽입하는 방법.
데이터를 삽입하여 원하는 문자열을 만들 수 있다.
- 문자열 **Formatting**을 위한 방법을 알아보자
 - **%** 기호 사용
 - **str.format()** method 사용

```
print( {}.format(값) )
```

```
print( {:포맷코드}.format(값) )
```

문자열 포매팅(Formatting)

- 문자열 포매팅의 필요성 ?
 - "현재 온도는 18도 입니다. "
시간이 지나서 20도가 되면... 다음 문장을 출력한다.
"현재 온도는 20도 입니다. "
 - 문자열에서 18을 20으로 바꾸어야 한다.
 - 이렇게 문자열 내의 특정한 값을 바꿔야 할 경우,
문자열 포매팅을 사용한다.

"현재 온도는 ●도 입니다. "



여기에 값을 삽입하여 출력

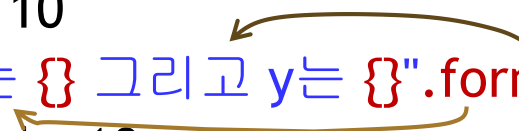
문자열 포매팅 - str.format() 사용

- str.format()
 - 문자열 자료형의 method(). 중괄호 {}를 사용.
 - 문자열 안에 중괄호 {}로 데이터가 들어갈 자리를 만들고, format() 함수의 인수로 삽입될 데이터를 지정해준다.
 - {}에 들어갈 데이터를 차례대로 지정.
 - {}의 위치에 데이터를 삽입해서 원하는 형식의 문자열을 만든다.

```
print( '{}'.format(값) )
```

- 예) 변수 이름과 변수 값 출력하기. 값 2개 삽입.

```
>>> x = 5; y = 10
>>> print( "x는 {} 그리고 y는 {}".format(x,y) )
x는 5 그리고 y는 10
```



- 문자열 'x는 그리고 y는 ' 안에 변수 값을 삽입.
- 문자열 안의 첫 괄호는 x 값, 둘째 괄호는 y 값으로 대체.

문자열 포매팅 - str.format() 사용

- str.format() 은 다음과 같은 형식을 갖는다
 - {} 중괄호에 차례대로 인수들이 mapping 된다.
 - 인수의 출력 형식을 지정할 수 있다. (optional)
- 삽입되는 개수(인수)만큼 문자열 안에 {}삽입.

'{ } { } ... { } '.format(var0, var1,..., varn) # n개 삽입

- 인수의 출력형식 지정하려면,

'{:출력형식}'.format(var)

문자열 포매팅 - str.format() 사용

- 출력 형식

“{:출력형식}” .format(var)

- { }안에 기호 :를 사용해서 포맷코드를 지정할 수 있다
- 문자열 포매팅은 문자열 포맷코드 없이 사용 가능.

- 문자열 포맷코드는 문자열에 삽입되는 값의 서식화에 사용

포맷코드	설명
{}	포맷코드 지정없이 사용가능. 어떤 값도 삽입 가능
{:s}	문자열(String) 삽입. 기본적으로 왼쪽 정렬
{:d}	정수(Integer) 삽입. 기본적으로 오른쪽 정렬
{:f}	실수(float) 삽입.
{:<}	왼쪽 정렬로 포함
{:>}	오른쪽 정렬로 포함

문자열 포매팅 - str.format() 사용

- 문자열에 변수의 값을 삽입하여 출력하기
 - 예) 문자열 '현재 온도는 도 입니다.' 안에 온도를 삽입하기
 - 문자열 안에 온도를 삽입하고 싶은 위치에 {}를 넣고,
 - format()의 인수로서 삽입할 값을 지정한다.

문자열 포매팅(값을 삽입해서 새 문자열 생성)을
print()함수의 인수로 지정, 출력해 보자.

```
>>> temp = 18
>>> print( "현재 온도는 {}도 입니다.".format(temp) )
현재 온도는 18도 입니다.
>>>
>>> temp = 20
>>> print( "현재 온도는 {}도 입니다.".format(temp) )
현재 온도는 20도 입니다.
>>>
```


문자열 포매팅 - str.format() 사용

온도를 삽입 할 위치 #여기에 {} 를 넣는다

문자열

“

0	1	2	3	4	5	6	7	8	9	10	11	12
현	재		온	도	는		도		입	니	다	.

”

```
>>> "현재 온도는 {}도 입니다.".format(18)
'현재 온도는 18도 입니다.'
```

문자열 포매팅으로 새로 구성된 문자열

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	”
“	현	재		온	도	는	1	8	도		입	니	다	.	

문자열 포매팅 - str.format() 사용

- 실수 서식 지정하기
 - 특정 서식에 맞춰 숫자를 출력할 때 사용.
 - 변환코드 f에 소수점 이하 자릿수를 지정 가능
.mf m은 소수점 이하 자릿수 지정
- 예) 포맷코드(출력 형식) 지정하기. 소수점 이하 2자리로 출력하기

`{ : . 2 f }` 서식 지정자

소수점 이하 자릿수 변환 코드

```
>>> print( "average = {}".format(57.467657) )  
average = 57.467657 #서식 지정없이 출력
```

```
print( "average = {:.2f}".format(57.467657))  
print( "average = {:.2f}".format(57.4))  
print( "average = {:.2f}".format(57))
```

☺ 실행 화면

```
average = 57.47  
average = 57.40  
average = 57.00
```

문자열 포매팅 - str.format() 사용

- 예) 실수를 서식없이 출력하기

```
>>> a = 1/3
>>> print( "1/3=", a ) # 식과 계산 값을 출력.
1/3= 0.3333333333333333
>>>
```

- 서식 지정하여 출력하기

```
>>> a = 1/3;
>>> print( "1/3= {}".format(a) ) #문자열에 값 그대로 삽입.
1/3= 0.3333333333333333
>>> print( "1/3= {:.3f}".format(a) ) #문자열 '1/3= '에
1/3= 0.333 # 소수점이하 3자리 값으로 삽입
>>>
>>> print( "{:.3f}".format(a) ) #서식지정하여 값 만 출력.
0.333
>>>
```

문자열 포매팅 - str.format() 사용

- 서식 정렬하기
 - 서식 지정자에 < 기호를 삽입하여 지정된 너비 내에서 결과 서식 항목을 **왼쪽 정렬**로 설정할 수 있다.
 - 서식 지정자에 > 기호를 삽입하여 지정된 너비 내에서 결과 서식 항목을 **오른쪽 정렬**로 설정할 수 있다.
- 예) 자릿수를 지정해서 출력하기

```
>>> "{:13}".format("시냇가에 심은 나무") # 왼쪽 정렬
'시냇가에 심은 나무 '
>>> "{:>13}".format("시냇가에 심은 나무") # 오른쪽 정렬
'  시냇가에 심은 나무'
<<<
```

← 13 →
시냇가에 심은 나무□□□
□□□시냇가에 심은 나무

문자열 포매팅 - str.format() 사용

- 예) 2개 이상의 값 넣기

- 첫 번째 { }는 format 함수의 첫 번째 인수 값으로 바뀌고
- 두 번째 { }는 format 함수의 두 번째 인수 값으로 바뀐다.

인수가 차례대로 mapping.

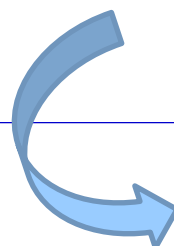
```
>>> print('{ } is a son of { }.'.format('Simon', 'John'))  
Simon is a son of John.  
>>>
```

```
>>> father="John"  
>>> son="Simon" # 차례대로 삽입  
>>> '{ } is a son of { }.'.format(son, father)  
'Simon is a son of John.'  
>>>
```

문자열 포매팅 - str.format() 사용

- 예) 문자열 포매팅으로 출력

```
>>> age = input("나이를 입력하시오: ")   
나이를 입력하시오: 24   
>>> age = int(age)  
>>> bornYear = 2022 - age + 1  
>>> print( str(bornYear) + '년생', str(age) + '살' )  
1999년생 24살  
>>>
```



```
>>> print( '{}년생 {}'.format(bornYear, age) )  
1999년생 24살  
>>>
```